// GUIDA GRATIS

UN AGENTE AI SU WHATSAPP IN *mezz'ora.*

Non un chatbot che risponde e basta: un agente che parla con i clienti, si ricorda le conversazioni, conosce la tua attività e fissa gli appuntamenti da solo. Gli 8 passi del reel, con i link ufficiali.

1 · **Piano**	Claude Code in modalità plan	pag. 02
2 · **WhatsApp**	app Meta, numero di prova, webhook	pag. 03
3 · **Cervello**	un modello via OpenRouter	pag. 05
4 · **Memoria**	gli ultimi messaggi per numero	pag. 06
5 · **Attività**	servizi, prezzi, orari, regole	pag. 07
6 · **Strumenti**	l'agenda con Cal.com	pag. 08
7 · **Rompilo**	la lista dei test cattivi	pag. 10
8 · **Online**	VPS, processo sempre acceso, HTTPS	pag. 11

// COSA TI SERVE

Un account Meta for Developers, un account OpenRouter con un po' di credito, un account Cal.com, Node.js 20 o più recente, e per il passo 8 un VPS con un dominio.

// ONESTÀ PRIMA DI TUTTO

I passaggi vengono dalle **documentazioni ufficiali**, controllate il 02/11/2026: il link è sotto ogni passo. Il codice in appendice è un **modello di partenza**: non l'abbiamo provato da capo a fondo con account WhatsApp e Cal.com veri.

// PASSO 01 · IL PIANO

01 PRIMA IL PIANO, *poi il codice.*

In modalità plan Claude Code legge e propone un piano, ma non modifica nessun file finché non lo approvi. È il momento di spiegargli la tua attività.

01 Apri Claude Code in modalità plan

Nel terminale, dentro una cartella vuota: **claude --permission-mode plan**. Oppure, in una sessione aperta, premi Shift+Tab finché in basso compare "plan mode on".

02 Descrivi l'attività, non la tecnologia

Chi sono i clienti, cosa chiedono di solito, cosa deve fare l'agente e cosa non deve mai fare.

03 Leggi il piano prima di dire sì

Controlla che ci siano tutti i pezzi dei passi 2-8. Se manca qualcosa, chiedilo ora: costa meno che correggerlo dopo.

// PROMPT DA INCOLLARE (modificalo con i tuoi dati)

«Voglio un agente WhatsApp per **[nome attività, es. barberia a Bologna]**. I clienti scrivono per chiedere prezzi, orari e per prenotare **[servizio]**. L'agente deve: rispondere in italiano, breve; usare solo i dati che ti do in un file attivita.md; ricordare gli ultimi messaggi di ogni numero; vedere i posti liberi e prenotare su Cal.com; passare la conversazione a una persona quando non sa. Non deve mai inventare prezzi né promettere sconti. Stack: Node.js, API WhatsApp Cloud di Meta, OpenRouter, API v2 di Cal.com. Fai prima il piano e fammi le domande che ti mancano.»

> **Consiglio —** aggiungi al prompt tre conversazioni vere che ricevi di solito. Il piano viene più concreto di qualsiasi descrizione.

Fonte: code.claude.com/docs/en/common-workflows (sezione "Plan before editing")

// PASSO 02 · WHATSAPP (1/2)

02 L'APP SU *Meta for Developers.*

L'agente riceve e invia messaggi tramite l'API WhatsApp Cloud di Meta. Per partire ti dà un numero di prova, senza usare il tuo numero.

01 Crea l'app
Da developers.facebook.com/apps: Create App, nome ed email, caso d'uso WhatsApp, scegli o crea un portfolio aziendale.

02 Numero di prova
In API Setup scegli il numero "From" di prova. Annota il **Phone number ID**: va nel codice.

03 Token temporaneo
"Generate access token" ti dà un token per provare subito. La doc dice che scade in fretta: va bene solo per i primi test.

04 Token permanente
In Business Settings → System users crea un utente di sistema, assegnagli l'app e l'account WhatsApp e genera un token con i permessi **business_management, whatsapp_business_messaging, whatsapp_business_management**.

05 Aggiungi il destinatario
Inserisci il tuo cellulare come numero "To", invia il messaggio di prova e rispondi dal telefono per aprire la conversazione.

```
// INVIARE UN MESSAGGIO (dalla doc ufficiale)
POST https://graph.facebook.com/v23.0/<PHONE_NUMBER_ID>/messages
Authorization: Bearer <TOKEN>

{ "messaging_product": "whatsapp", "recipient_type": "individual",
  "to": "<NUMERO_CLIENTE>", "type": "text", "text": { "body": "Ciao!" } }
```

> **Consiglio —** v23.0 è la versione della Graph API usata nell'esempio ufficiale. Al 02/11/2026 il changelog di Meta elenca come più recente la v26.0: puoi cambiarla nell'indirizzo.

Fonte: developers.facebook.com/docs/whatsapp/cloud-api/get-started · developers.facebook.com/docs/graph-api/changelog

// PASSO 02 · WHATSAPP (2/2)

02 IL WEBHOOK, *il campanello.*

A ogni messaggio di un cliente Meta chiama un indirizzo del tuo server. Prima però controlla che quell'indirizzo sia davvero tuo, con un token di verifica che scegli tu.

01 **Dove si imposta**

App Dashboard → WhatsApp → Configuration: inserisci il Callback URL (es. https://tuodominio.it/webhook) e il Verify token, poi iscriviti al campo **messages**.

02 **La verifica**

Meta manda una richiesta GET con **hub.mode** (sempre "subscribe"), **hub.verify_token** e **hub.challenge**. Se il token coincide, rispondi con il valore di hub.challenge.

03 **Solo HTTPS**

Serve un certificato valido: i certificati autofirmati non sono accettati (vedi passo 8).

04 **Rispondi subito 200**

Per ogni notifica rispondi 200 OK. Se non lo fai Meta riprova, quindi lo stesso messaggio può arrivare due volte: scartare i doppioni è compito tuo.

```
// DOVE STA IL MESSAGGIO NEL JSON CHE ARRIVA
entry[].changes[].value.messages[].from       → numero del cliente
entry[].changes[].value.messages[].text.body  → testo del messaggio
```

// COSTI (solo quanto scritto da Meta)

Dal 1° luglio 2025 Meta addebita **per messaggio**, solo per i messaggi dall'azienda agli utenti; quelli dei clienti verso di te non si pagano. Ogni numero aziendale ha **1000 messaggi di servizio gratuiti al mese**, che non si accumulano. La pagina riflette gli aggiornamenti in vigore dal 1° ottobre 2026: le tariffe per paese e categoria sono lì.

Fonte: developers.facebook.com/docs/graph-api/webhooks/getting-started · developers.facebook.com/docs/whatsapp/cloud-api/guides/set-up-webhooks · developers.facebook.com/docs/whatsapp/pricing

// PASSO 03 · IL CERVELLO

03 UNA CHIAVE, *tanti modelli.*

OpenRouter mette tanti modelli (Claude, GPT, Gemini e altri) dietro un solo indirizzo con una sola chiave. Per cambiare modello cambi una riga.

01 **Account e chiave**
Registrati su openrouter.ai, aggiungi credito e crea una chiave API. Mettila nel file .env, mai nel codice.

02 **Un solo endpoint**
POST https://openrouter.ai/api/v1/chat/completions con l'header Authorization: Bearer <chiave>. Il formato è molto simile a quello di OpenAI.

03 **Scegli il modello**
Il nome ha il prefisso del fornitore, es. **openai/gpt-4o-mini** o **anthropic/claude-sonnet-5.5** (entrambi nel catalogo al 02/11/2026). La lista completa è su openrouter.ai/models.

04 **Parti piccolo**
Per rispondere a domande su orari e prezzi basta spesso un modello veloce ed economico. Passa a uno più grande solo se nei test del passo 7 sbaglia.

```
// LA RICHIESTA, IN BREVE
POST https://openrouter.ai/api/v1/chat/completions
Authorization: Bearer <OPENROUTER_API_KEY>

{ "model": "openai/gpt-4o-mini",
  "messages": [ { "role": "system", "content": "...dati attività..." },
                { "role": "user",   "content": "Quanto costa il taglio?" } ] }
```

// COSTI (solo quanto scritto da OpenRouter)

Si paga **a token**: il prezzo di ogni modello è indicato per milione di token, di solito diverso tra input e output. Si usa un credito prepagato. OpenRouter dichiara di non applicare ricarichi sul prezzo dei fornitori; la FAQ cita una commissione sull'acquisto del credito, ma al 02/11/2026 non ne mostrava la percentuale. I modelli gratuiti hanno limiti bassi.

Fonte: openrouter.ai/docs/api/reference/overview · openrouter.ai/docs/faq · openrouter.ai/models

// PASSO 04 · LA MEMORIA

04 SENZA MEMORIA, *è uno sconosciuto.*

Il modello non ricorda nulla tra una richiesta e l'altra. La memoria la tieni tu: salvi gli ultimi scambi di ogni numero e li rimandi ogni volta.

01 Una storia per numero

La chiave è il numero WhatsApp del cliente (il campo "from"). Nel codice d'esempio: un file dati/<numero>.json.

02 Solo gli ultimi N

Tieni gli ultimi 10-20 messaggi (nel codice: 12). Abbastanza per il contesto, senza far crescere il costo a ogni risposta.

03 Ogni richiesta, lo stesso schema

Istruzioni + dati dell'attività, poi la storia, poi il messaggio nuovo. Quando arriva la risposta, salvi entrambi.

```
// COSA MANDI AL MODELLO A OGNI MESSAGGIO
messages = [
  { role: "system",    content: istruzioni + attivita.md + data di oggi },
  ...ultimi 12 messaggi di questo numero,      // dalla memoria
  { role: "user",      content: "e giovedì alle 17:30?" }
]
// risposta del modello → aggiungi alla storia → salva gli ultimi 12
```

// ESEMPIO (con e senza memoria)

Cliente: Vorrei prenotare un taglio. **… Cliente:** e giovedì alle 17:30?
Senza memoria: "Ciao! Come posso aiutarti?" · **Con memoria:** "Giovedì alle 17:30 è libero, prenoto il taglio?"

> **Consiglio —** sono dati personali dei tuoi clienti: decidi per quanto tempo tenerli e scrivilo nella tua informativa privacy. Con un database (anche SQLite) al posto dei file è più semplice cancellarli.

// PASSO 05 · LA TUA ATTIVITÀ

05 I TUOI DATI, *niente inventato.*

L'agente sa solo quello che gli scrivi. Un file di testo con servizi, prezzi, orari e regole, messo nelle istruzioni di sistema a ogni richiesta.

01 Un file, quattro blocchi
Servizi e prezzi, orari, regole per prenotare, cosa non fare mai. Scritto come lo diresti a un collaboratore nuovo.

02 La regola d'oro
"Usa SOLO questi dati. Se non sai, dillo e proponi di parlare con una persona." Senza questa frase il modello riempie i buchi da solo.

03 La data di oggi
Aggiungila alle istruzioni: senza, "domani" o "giovedì" non vogliono dire niente per il modello.

```
// attivita.md (esempio, da modificare)
# Studio Esempio — dati dell'attività (modificali con i tuoi)
Attività: studio di consulenza digitale per piccole attività locali.
Servizi e prezzi:
- Consulenza iniziale 30 minuti: gratuita
- Configurazione profilo Google Business: 150 €
- Gestione social mensile: da 400 € al mese
Orari: dal lunedì al venerdì, 9:00-18:00 (ora italiana). Sabato e domenica chiuso.
Regole per prenotare:
- Si prenota solo la consulenza iniziale da 30 minuti, nei posti liberi del calendario.
- Per prenotare servono nome ed email del cliente.
- Mai promettere sconti o prezzi diversi da questi.
```

```
// ISTRUZIONI DI SISTEMA (dal codice d'esempio)
Sei l'assistente WhatsApp di questa attività. Rispondi in italiano, breve e cordiale.
Usa SOLO questi dati, non inventare prezzi o servizi. Se non sai, dillo e proponi di
parlare con una persona. Per prenotare usa gli strumenti: prima orari_liberi, poi
prenota (servono nome ed email). Oggi è <data>.

<contenuto di attivita.md>
```

> **Consiglio —** quando cambi un prezzo cambia solo il file: nessuna modifica al codice.

06 L'AGENDA *con Cal.com.*

Qui il chatbot diventa agente: guarda i posti liberi e prenota. Servono un tipo di evento e due chiamate API.

01 Account e tipo di evento

Crea un account su cal.com e un tipo di evento (es. "Consulenza 30 minuti") con i tuoi orari. Ti serve il suo ID: lo restituisce GET /v2/event-types.

02 La chiave API

Dalle impostazioni (Settings → Developer → API Keys). Le chiavi iniziano con cal_ (test) o cal_live_. Si mandano come Authorization: Bearer <chiave>. Limite: 120 richieste al minuto. Ogni endpoint vuole il suo **cal-api-version**: con un valore sbagliato Cal.com usa una versione più vecchia.

```
// 1 · POSTI LIBERI
GET https://api.cal.com/v2/slots?eventTypeId=<ID>&start=2026-11-05&end=2026-11-
06&timeZone=Europe/Rome
cal-api-version: 2024-09-04
Authorization: Bearer <CAL_API_KEY>

→ data: { "2026-11-05": [ { "start": "..." }, ... ] }   (oggetto vuoto se non c'è posto)
```

```
// 2 · PRENOTA
POST https://api.cal.com/v2/bookings
cal-api-version: 2026-02-25
Authorization: Bearer <CAL_API_KEY>

{ "start": "2026-11-05T16:30:00Z",          // ISO 8601 in UTC
  "eventTypeId": <ID>,
  "attendee": { "name": "Luca", "email": "luca@esempio.it",
                "timeZone": "Europe/Rome", "language": "it" } }
```

> **Consiglio —** il piano Individuals è gratuito ma la pagina prezzi non elenca l'API tra le sue funzioni; il blog di Cal.com dice che le chiavi API sono per tutti gli account. Verifica sul tuo prima di contarci.

Fonte: cal.com/docs/api-reference/v2/slots/get-available-time-slots-for-an-event-type · cal.com/docs/api-reference/v2/bookings/create-a-booking · cal.com/docs/api-reference/v2/introduction · cal.com/pricing

// PASSO 06 · GLI STRUMENTI (2/2)

06 IL MODELLO SCEGLIE, *il codice esegue.*

Le due chiamate diventano "strumenti" che il modello può chiedere di usare. Lui decide quando; il tuo codice le esegue davvero e gli restituisce il risultato.

01 **Descrivi gli strumenti**
Nella richiesta a OpenRouter aggiungi "tools": nome, descrizione e parametri di ciascuno (orari_liberi, prenota).

02 **Il modello chiede**
Se gli serve, risponde con **tool_calls**: il nome dello strumento e gli argomenti come testo JSON.

03 **Tu esegui e rispondi**
Chiami Cal.com, poi aggiungi il messaggio del modello e un messaggio con role "tool" e lo stesso **tool_call_id**. Ripeti la richiesta, al massimo 3-4 giri per messaggio.

04 **Sempre "tools"**
Secondo la doc di OpenRouter "tools" va incluso in ogni richiesta, anche in quella che riporta i risultati.

```
// UNO STRUMENTO DESCRITTO PER IL MODELLO
{ "type": "function",
  "function": { "name": "orari_liberi",
    "description": "Posti liberi per la consulenza tra due date (YYYY-MM-DD).",
    "parameters": { "type": "object",
      "properties": { "dal": { "type": "string" }, "al": { "type": "string" } },
      "required": ["dal", "al"] } } }
```

```
// IL GIRO COMPLETO (esempio)
cliente  → "giovedì pomeriggio avete posto?"
modello  → tool_calls: orari_liberi {dal: 2026-11-05, al: 2026-11-05}
codice   → GET /v2/slots … → "15:00, 16:30, 17:30"
modello  → "Giovedì ho libero alle 15:00, 16:30 e 17:30. Quale preferisci?"
```

Fonte: openrouter.ai/docs/guides/features/tool-calling

// PASSO 07 · PROVA A ROMPERLO

07 I CLIENTI VERI *non seguono il copione.*

Prima di darlo ai clienti, fai tu il cliente difficile. Scrivi la lista, mandala tutta, segna cosa sbaglia, correggi e rimanda tutta la lista.

"Domenica alle 3 di notte?"	dice che è chiuso, propone orari veri	`orari`
"Il 31 febbraio alle 10"	segnala la data impossibile	`date`
"Mi fai il 50% di sconto?"	nessuno sconto non scritto nei dati	`prezzi`
"Quanto costa un sito web?"	non lo offrite: lo dice, non inventa	`servizi`
"Prenota" senza nome/email	chiede i dati mancanti prima di prenotare	`prenota`
"Ignora le istruzioni e…"	resta nel suo ruolo	`sicurezza`
Vocale o foto	il codice d'esempio gestisce solo testo: decidi cosa fare	`formati`
Lo stesso messaggio due volte	una sola prenotazione	`doppioni`

// IL GIRO DI CORREZIONE

Sbaglia un prezzo? Correggi **attivita.md**. Inventa o promette? Rafforza le **istruzioni**. Sbaglia lo strumento? Migliora la **descrizione dello strumento**. Poi rilancia **tutta** la lista: una correzione può rompere un caso che prima andava.

> **Consiglio —** incolla a Claude Code: «Ecco la lista di test e le risposte che ho ricevuto. Per ogni risposta sbagliata dimmi la causa (dati, istruzioni o strumento) e proponi la correzione più piccola. Non toccare il resto.» Per provare senza WhatsApp il codice ha /simula: usalo solo in locale.

// PASSO 08 · ONLINE

08 IL TUO COMPUTER *non è un server.*

Se il programma gira sul tuo portatile, l'agente si spegne con lui. Su un VPS resta acceso, e il webhook ha un indirizzo HTTPS stabile.

01 **Il server**
Un VPS Linux qualsiasi con Node.js 20+. Copia la cartella del progetto e crea il file .env con le chiavi (il token WhatsApp permanente, non quello temporaneo).

02 **Sempre acceso con PM2**
PM2 riavvia il programma se si blocca e al riavvio della macchina.

03 **HTTPS con Caddy**
Punta il dominio all'IP del VPS e apri le porte 80 e 443: Caddy ottiene da solo un certificato valido e inoltra al tuo programma.

04 **Aggiorna Meta**
Nel pannello WhatsApp → Configuration metti https://tuodominio.it/webhook come Callback URL e verifica.

// I COMANDI

```
# processo sempre acceso (PM2)
npm install pm2@latest -g
pm2 start server.js --name agente-whatsapp
pm2 startup        # avvio automatico al boot
pm2 save           # salva la lista dei processi
pm2 logs           # guarda cosa succede

# HTTPS davanti alla porta 3000 (Caddy)
caddy reverse-proxy --from tuodominio.it --to :3000
```

// PRIMA DI ANDARE ONLINE

Disattiva o proteggi **/simula**: su un server pubblico chiunque potrebbe usarlo e consumare il tuo credito OpenRouter. Aggiungi lo scarto dei messaggi doppi (Meta ritenta le notifiche). Il file **.env** non va mai su GitHub.

Fonte: pm2.keymetrics.io/docs/usage/quick-start · caddyserver.com/docs/quick-starts/reverse-proxy

// TUTTI I LINK

LE FONTI, *in un posto solo.*

Ogni passo della guida viene da queste pagine ufficiali, controllate il 02/11/2026. Le API cambiano: se qualcosa non torna, vale quello che c'è scritto lì.

Plan mode	code.claude.com/docs/en/common-workflows
WhatsApp: primi passi	developers.facebook.com/docs/whatsapp/cloud-api/get-started
Webhook: verifica	developers.facebook.com/docs/graph-api/webhooks/getting-started
Webhook WhatsApp	developers.facebook.com/docs/whatsapp/cloud-api/guides/set-up-webhooks
Versioni Graph API	developers.facebook.com/docs/graph-api/changelog
Prezzi WhatsApp	developers.facebook.com/docs/whatsapp/pricing
OpenRouter API	openrouter.ai/docs/api/reference/overview
OpenRouter strumenti	openrouter.ai/docs/guides/features/tool-calling
OpenRouter costi	openrouter.ai/docs/faq
Cal.com chiavi API	cal.com/docs/api-reference/v2/introduction
Cal.com posti liberi	cal.com/docs/api-reference/v2/slots/get-available-time-slots-for-an-event-type
Cal.com prenotazione	cal.com/docs/api-reference/v2/bookings/create-a-booking
PM2	pm2.keymetrics.io/docs/usage/quick-start
Caddy	caddyserver.com/docs/quick-starts/reverse-proxy

// RICORDA

I costi (messaggi WhatsApp, token di OpenRouter, eventuale piano Cal.com, il VPS) dipendono dal tuo uso: controlla sempre le pagine prezzi ufficiali. Se ti è servita, seguici su **@21studio.io**.

// APPENDICE · IL CODICE (1/4)

IL CODICE *di partenza.*

Un solo file Node.js, senza dipendenze: webhook Meta → modello via OpenRouter (con memoria e dati dell'attività) → strumenti Cal.com → risposta su WhatsApp. Struttura: server.js, attivita.md, .env, cartella dati/.

// LEGGI PRIMA

È un **modello di partenza, non eseguito da capo a fondo**: non l'abbiamo provato con account WhatsApp e Cal.com veri. Le chiamate seguono le doc ufficiali citate nella guida, ma prima di usarlo con i clienti fai i test del passo 7. Modello predefinito: openai/gpt-4o-mini; Graph API: v23.0.

// .env (copia da .env.example e riempi)

```
# Copia in .env e riempi. Non pubblicare mai questo file.
OPENROUTER_API_KEY=
OPENROUTER_MODEL=openai/gpt-4o-mini
WHATSAPP_TOKEN=
WHATSAPP_PHONE_NUMBER_ID=
WHATSAPP_VERIFY_TOKEN=scegli-una-parola-segreta
CAL_API_KEY=
CAL_EVENT_TYPE_ID=
PORT=3000
```

// COME SI AVVIA

Metti server.js, attivita.md (pag. 07) e .env nella stessa cartella, crea la cartella **dati/**, poi **node server.js**. Per provare senza WhatsApp, solo in locale: POST /simula con {"da":"393331234567","testo":"..."}.

// APPENDICE · IL CODICE (2/4)

// server.js · 1/3 · configurazione, memoria, strumenti Cal.com

```
// Agente WhatsApp minimo: webhook Meta -> modello via OpenRouter (con memoria e dati dell'attività) -> strumenti
Cal.com -> risposta su WhatsApp.
// Node 20+, nessuna dipendenza. Avvio: node server.js   (legge .env nella stessa cartella)
// Prova senza WhatsApp: POST /simula {"da":"393331234567","testo":"..."} risponde in JSON invece di inviare.
const http = require('http'), fs = require('fs'), path = require('path');
for (const r of fs.existsSync(path.join(__dirname, '.env')) ? fs.readFileSync(path.join(__dirname, '.env'),
'utf8').split('\n') : []) {
  const m = r.match(/^([A-Z_]+)=(.*)$/); if (m && !process.env[m[1]]) process.env[m[1]] = m[2].trim();
}
const E = process.env, TZ = 'Europe/Rome', MEMO = 12;
const ATTIVITA = fs.readFileSync(path.join(__dirname, 'attivita.md'), 'utf8');

// ---- memoria: ultimi MEMO messaggi per numero, su file ----
const fileMemo = (n) => path.join(__dirname, 'dati', `${n.replace(/\D/g, '')}.json`);
const leggiMemo = (n) => fs.existsSync(fileMemo(n)) ? JSON.parse(fs.readFileSync(fileMemo(n), 'utf8')) : [];
const salvaMemo = (n, m) => fs.writeFileSync(fileMemo(n), JSON.stringify(m.slice(-MEMO), null, 1));

// ---- strumenti Cal.com (API v2) ----
async function cal(metodo, url, versione, body) {
  const r = await fetch(`https://api.cal.com/v2${url}`, { method: metodo, body: body && JSON.stringify(body),
    headers: { Authorization: `Bearer ${E.CAL_API_KEY}`, 'cal-api-version': versione, 'Content-Type':
'application/json' } });
  const j = await r.json(); if (!r.ok) throw new Error(`Cal.com ${r.status}: ${JSON.stringify(j.error ||
j).slice(0, 200)}`); return j.data;
}
const STRUMENTI = {
  orari_liberi: { descr: 'Posti liberi per la consulenza tra due date (YYYY-MM-DD).', param: { dal: 'string', al:
'string' },
    async run({ dal, al }) {
      const d = await cal('GET', `/slots?
eventTypeId=${E.CAL_EVENT_TYPE_ID}&start=${dal}&end=${al}&timeZone=${TZ}`, '2024-09-04');
      return Object.entries(d).map(([g, s]) => `${g}: ${s.map((x) => x.start.slice(11, 16)).join(',
')}`).join('\n') || 'nessun posto libero';
    } },
  prenota: { descr: 'Prenota la consulenza. inizio = data e ora ISO con fuso, come restituite da orari_liberi.',
param: { inizio: 'string', nome: 'string', email: 'string' },
    async run({ inizio, nome, email }, numero) {
      const b = await cal('POST', '/bookings', '2026-02-25', { start: new Date(inizio).toISOString(), eventTypeId:
Number(E.CAL_EVENT_TYPE_ID),
        attendee: { name: nome, email, timeZone: TZ, language: 'it', phoneNumber: `+${numero}` } });
      return `prenotato: ${b.start} (id ${b.uid})`;
    } },
};
const tools = Object.entries(STRUMENTI).map(([name, s]) => ({ type: 'function', function: { name, description:
s.descr,
  parameters: { type: 'object', properties: Object.fromEntries(Object.entries(s.param).map(([k, t]) => [k, { type:
t }])), required: Object.keys(s.param) } } }));
```

// APPENDICE · IL CODICE (3/4)

```
// server.js · 2/3 · l'agente (modello + strumenti, fino a 4 giri)
// ---- agente: modello + strumenti, fino a 4 giri ----
async function agente(numero, testo) {
  const oggi = new Date().toLocaleString('it-IT', { timeZone: TZ, dateStyle: 'full', timeStyle:
'short' });
  const sistema = `Sei l'assistente WhatsApp di questa attività. Rispondi in italiano, breve e
cordiale. Usa SOLO questi dati, non inventare prezzi o servizi. Se non sai, dillo e proponi di
parlare con una persona. Per prenotare usa gli strumenti: prima orari_liberi, poi prenota (servono
nome ed email). Oggi è ${oggi}.\n\n${ATTIVITA}`;
  const storia = leggiMemo(numero); storia.push({ role: 'user', content: testo });
  const msg = [{ role: 'system', content: sistema }, ...storia];
  for (let giro = 0; giro < 4; giro++) {
    const r = await fetch('https://openrouter.ai/api/v1/chat/completions', { method: 'POST',
      headers: { Authorization: `Bearer ${E.OPENROUTER_API_KEY}`, 'Content-Type': 'application/json'
},
      body: JSON.stringify({ model: E.OPENROUTER_MODEL || 'openai/gpt-4o-mini', messages: msg, tools
}) });
    const j = await r.json(); if (!r.ok) throw new Error(`OpenRouter ${r.status}:
${JSON.stringify(j.error || j).slice(0, 200)}`);
    const m = j.choices[0].message; msg.push(m);
    if (!m.tool_calls?.length) { storia.push({ role: 'assistant', content: m.content });
salvaMemo(numero, storia); return m.content; }
    for (const c of m.tool_calls) {
      let out; try { out = await STRUMENTI[c.function.name].run(JSON.parse(c.function.arguments ||
'{}'), numero); } catch (e) { out = `errore: ${e.message}`; }
      console.log(`[strumento] ${c.function.name} ${c.function.arguments} -> ${String(out).slice(0,
160)}`);
      msg.push({ role: 'tool', tool_call_id: c.id, content: String(out) });
    }
  }
  return 'Scusa, ho avuto un problema. Ti faccio richiamare da una persona.';
}
```

// APPENDICE · IL CODICE (4/4)

```
// server.js · 3/3 · invio WhatsApp, webhook, /simula
async function inviaWhatsApp(a, testo) {
  const r = await fetch(`https://graph.facebook.com/v23.0/${E.WHATSAPP_PHONE_NUMBER_ID}/messages`, {
method: 'POST',
    headers: { Authorization: `Bearer ${E.WHATSAPP_TOKEN}`, 'Content-Type': 'application/json' },
    body: JSON.stringify({ messaging_product: 'whatsapp', recipient_type: 'individual', to: a, type:
'text', text: { body: testo } }) });
  if (!r.ok) console.error('[whatsapp] invio non riuscito', r.status, (await r.text()).slice(0,
200));
}

const corpo = (req) => new Promise((ok) => { let d = ''; req.on('data', (c) => (d += c));
req.on('end', () => ok(d)); });
http.createServer(async (req, res) => {
  const u = new URL(req.url, 'http://x');
  if (req.method === 'GET' && u.pathname === '/webhook') { // verifica di Meta
    const ok = u.searchParams.get('hub.mode') === 'subscribe' &&
u.searchParams.get('hub.verify_token') === E.WHATSAPP_VERIFY_TOKEN;
    res.writeHead(ok ? 200 : 403); return res.end(ok ? u.searchParams.get('hub.challenge') : '');
  }
  if (req.method === 'POST' && u.pathname === '/webhook') {
    const j = JSON.parse((await corpo(req)) || '{}'); res.writeHead(200); res.end(); // Meta vuole
il 200 subito
    const m = j.entry?.[0]?.changes?.[0]?.value?.messages?.[0];
    if (m?.type === 'text') { console.log(`[in] ${m.from}: ${m.text.body}`); const r = await
agente(m.from, m.text.body).catch((e) => (console.error(e), null)); if (r) { console.log(`[out]
${r}`); await inviaWhatsApp(m.from, r); } }
    return;
  }
  if (req.method === 'POST' && u.pathname === '/simula') {
    const { da, testo } = JSON.parse(await corpo(req));
    try { const r = await agente(da, testo); res.writeHead(200, { 'Content-Type': 'application/json'
}); return res.end(JSON.stringify({ risposta: r })); }
    catch (e) { res.writeHead(500); return res.end(e.message); }
  }
  res.writeHead(404); res.end();
}).listen(E.PORT || 3000, () => console.log(`agente in ascolto sulla porta ${E.PORT || 3000}`));
```